Companion to Bird & de Moor, Algebra of Programming

§10. Greedy Algorithms

§10.1. Theory

(10.1a)

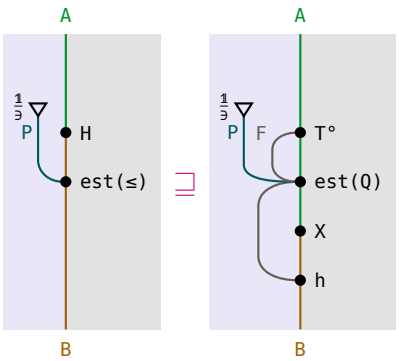
h, T, ≤, H, M as in M ≜ $\frac{H}{\ni}$ est(≤) ; additionally Q a **connected** preorder on the sets $\frac{T^\circ}{\ni}$ returns, so that $\frac{T^\circ}{\ni}$ est(Q) is entire.

Map(h); h Pres ≤; ≤² ⊑ ≤; ThinCondition(T,h,≤,Q) Theorem 10.1

⇒ (μX : $\frac{T^\circ}{\ni}$ est(Q) F(X) h) ⊑ M

the same optimum reached by keeping ONE decomposition at each step, so that no set is ever carried and the recursion runs on values alone

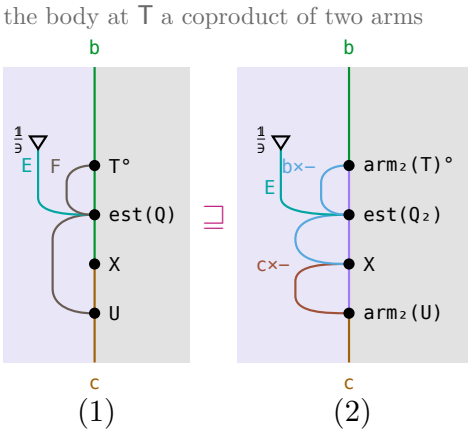
H≜(T)°(h) — ≜ H, Q



(1) (2)

1	M	
2	⊒ $\frac{T^\circ}{\ni}$ est(Q) F(X) h	

the body at T a coproduct of two arms

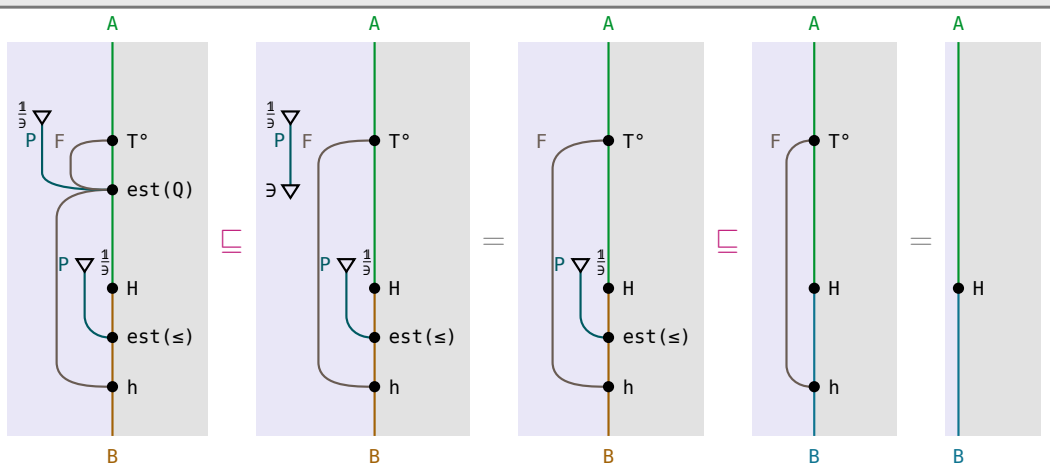


(1) (2)

1	$\frac{T^\circ}{\ni}$ est(Q) F(X) U	Proposition 10.1, V₂V₁°=0
2	⊒ $\frac{arm_2(T)^\circ}{\ni}$ est(Q₂) (X × 𝟙) arm₂(U)	

$\frac{T^\circ}{\ni}$ est(Q) F(M) h ⊑ H (10.1b)

taking the input apart in one Q-extreme way, solving the parts by M and assembling by h returns only what H returns



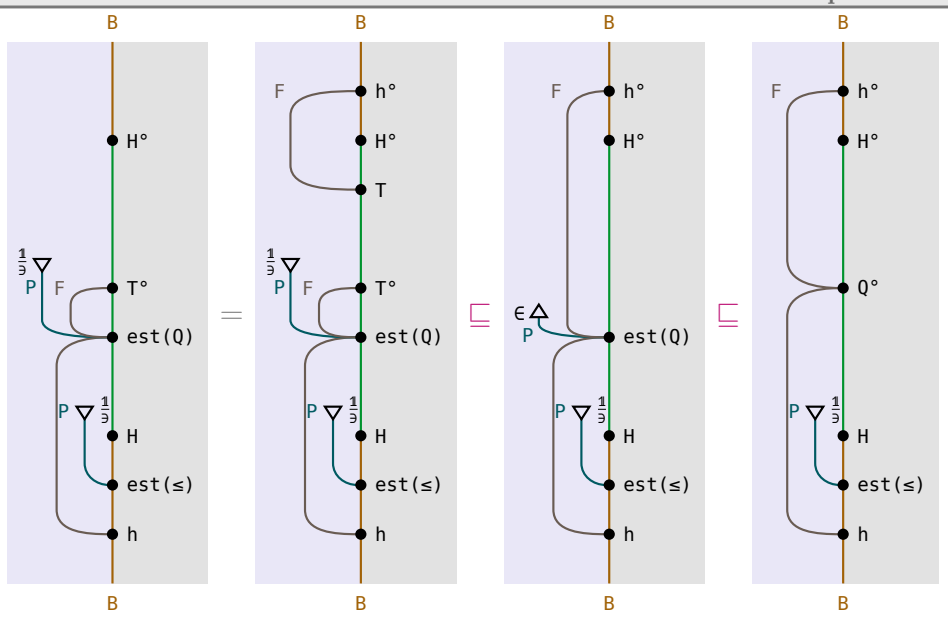
(1) (2) (3) (4) (5)

1	$\frac{T^\circ}{\ni}$ est(Q) F($\frac{H}{\ni}$ est(≤)) h	est(Q) ⊑ ∋
2	⊒ $\frac{T^\circ}{\ni}$ ∋ F($\frac{H}{\ni}$ est(≤)) h	$\frac{R}{\ni}$ ∋ = R
3	= T° F($\frac{H}{\ni}$ est(≤)) h	$\frac{H}{\ni}$ est(≤) ⊑ H
4	⊑ T° F(H) h	T° F(H) h = H
5	= H	

Map(h); h Pres ≤; ≤² ⊑ ≤; ThinCondition(T,h,≤,Q) (10.1c)

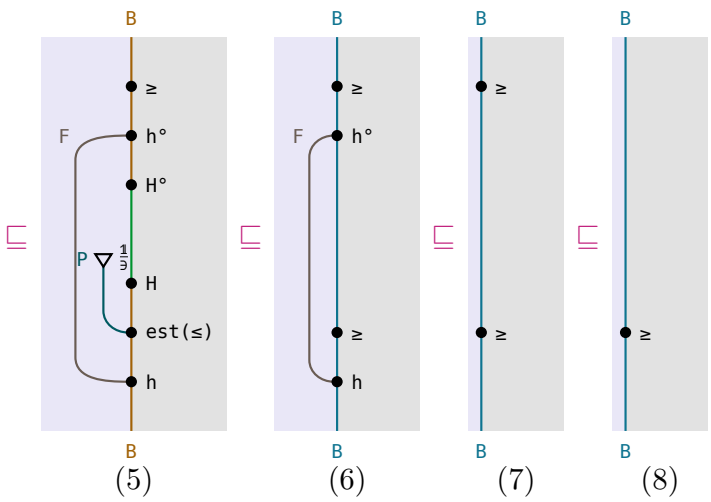
⇒ H° $\frac{T°}{∋}$ est(Q) F(M) h ⊑ ≥

H° followed by the greedy body at **M** is **⊑≥**: an answer of the body is never worse than an answer of **H** to the same input



(1) (2) (3) (4)

1	H° $\frac{T°}{∋}$ est(Q) F($\frac{H}{∋}$ est(≤)) h	H° = h° F(H°) T
2	= h° F(H°) T $\frac{T°}{∋}$ est(Q) F($\frac{H}{∋}$ est(≤)) h	T $\frac{T°}{∋}$ ⊑ ∈
3	⊑ h° F(H°) ∈ est(Q) F($\frac{H}{∋}$ est(≤)) h	∈ est(Q) ⊑ Q°
4	⊑ h° F(H°) Q° F($\frac{H}{∋}$ est(≤)) h	ThinCondition(T,h,≤,Q) ⇒ h° F(H°) Q° ⊑ ≥ h° F(H°)



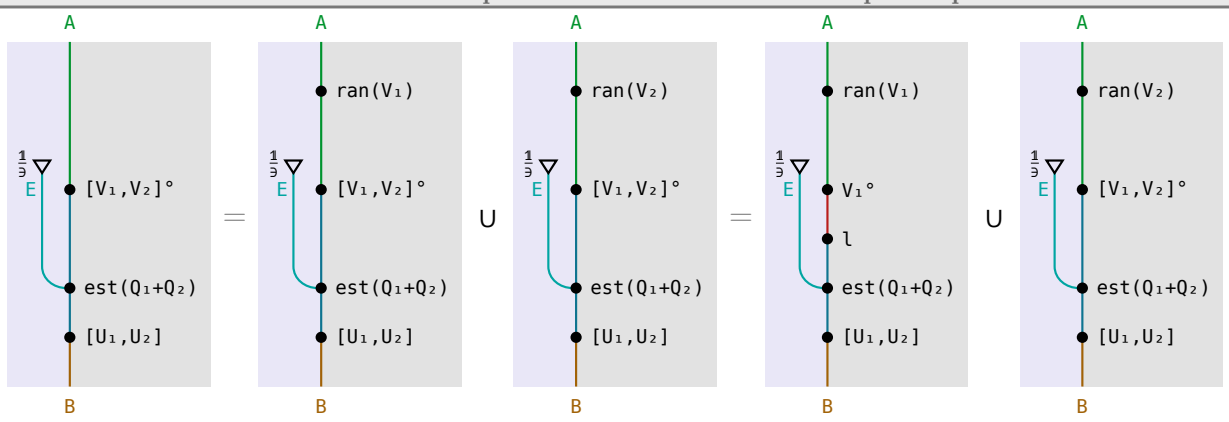
(5) (6) (7) (8)

5	⊑ ≥ h° F(H° $\frac{H}{∋}$ est(≤)) h	H° $\frac{H}{∋}$ est(≤) ⊑ ≥
6	⊑ ≥ h° F(≥) h	Map(h); h Pres ≤ ⇒ h° F(≥) h ⊑ ≥
7	⊑ ≥²	≤² ⊑ ≤ ⇒ ≥² ⊑ ≥
8	⊑ ≥	

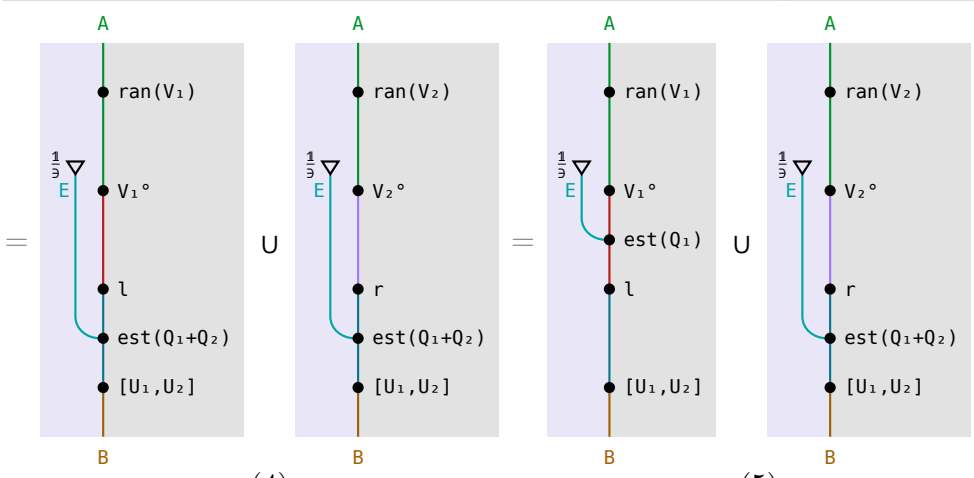
Proposition 10.1

$$V_2\ V_1{}^\circ = \emptyset \Rightarrow \frac{[V_1,V_2]^\circ}{\ni}\ est(Q_1 + Q_2)\ [U_1,U_2] = (ran(V_1)\ \frac{V_1{}^\circ}{\ni}\ est(Q_1)\ U_1) \cup (ran(V_2)\ \frac{V_2{}^\circ}{\ni}\ est(Q_2)\ U_2)$$

when V_1 and V_2 have disjoint ranges, the greedy step over $[V_1,V_2]$ runs the V_1 step on inputs V_1 reaches and the V_2 step on inputs V_2 reaches



1	$\frac{[V_1,V_2]^\circ}{\ni}\ est(Q_1 + Q_2)\ [U_1,U_2]$	$\frac{[V_1,V_2]^\circ}{\ni}\ est(Q_1 + Q_2)\ [U_1,U_2] = (ran(V_1)\ \frac{[V_1,V_2]^\circ}{\ni}\ est(Q_1 + Q_2)\ [U_1,U_2]) \cup (ran(V_2)\ \frac{[V_1,V_2]^\circ}{\ni}\ est(Q_1 + Q_2)\ [U_1,U_2])$
2	$= (ran(V_1)\ \frac{[V_1,V_2]^\circ}{\ni}\ est(Q_1 + Q_2)\ [U_1,U_2]) \cup (ran(V_2)\ \frac{[V_1,V_2]^\circ}{\ni}\ est(Q_1 + Q_2)\ [U_1,U_2])$	$V_2\ V_1{}^\circ = \emptyset \Rightarrow ran(V_1)\ \frac{[V_1,V_2]^\circ}{\ni} = ran(V_1)\ \frac{V_1{}^\circ}{\ni}\ P(l)$
3	$= (ran(V_1)\ \frac{V_1{}^\circ}{\ni}\ P(l)\ est(Q_1 + Q_2)\ [U_1,U_2]) \cup (ran(V_2)\ \frac{[V_1,V_2]^\circ}{\ni}\ est(Q_1 + Q_2)\ [U_1,U_2])$	$V_2\ V_1{}^\circ = \emptyset \Rightarrow ran(V_2)\ \frac{[V_1,V_2]^\circ}{\ni} = ran(V_2)\ \frac{V_2{}^\circ}{\ni}\ P(r)$



4	$= (ran(V_1)\ \frac{V_1{}^\circ}{\ni}\ P(l)\ est(Q_1 + Q_2)\ [U_1,U_2]) \cup (ran(V_2)\ \frac{V_2{}^\circ}{\ni}\ P(r)\ est(Q_1 + Q_2)\ [U_1,U_2])$	$P(l)\ est(Q_1 + Q_2) = est(Q_1)\ l$
5	$= (ran(V_1)\ \frac{V_1{}^\circ}{\ni}\ est(Q_1)\ l\ [U_1,U_2]) \cup (ran(V_2)\ \frac{V_2{}^\circ}{\ni}\ P(r)\ est(Q_1 + Q_2)\ [U_1,U_2])$	$P(r)\ est(Q_1 + Q_2) = est(Q_2)\ r$
6	$= (ran(V_1)\ \frac{V_1{}^\circ}{\ni}\ est(Q_1)\ l\ [U_1,U_2]) \cup (ran(V_2)\ \frac{V_2{}^\circ}{\ni}\ est(Q_2)\ r\ [U_1,U_2])$	$l\ [U_1,U_2] = U_1$
7	$= (ran(V_1)\ \frac{V_1{}^\circ}{\ni}\ est(Q_1)\ U_1) \cup (ran(V_2)\ \frac{V_2{}^\circ}{\ni}\ est(Q_2)\ r\ [U_1,U_2])$	$r\ [U_1,U_2] = U_2$
8	$= (ran(V_1)\ \frac{V_1{}^\circ}{\ni}\ est(Q_1)\ U_1) \cup (ran(V_2)\ \frac{V_2{}^\circ}{\ni}\ est(Q_2)\ U_2)$	

§10.2. The detab-entab problem

§10.2.1. Detab

(10.2.1a)

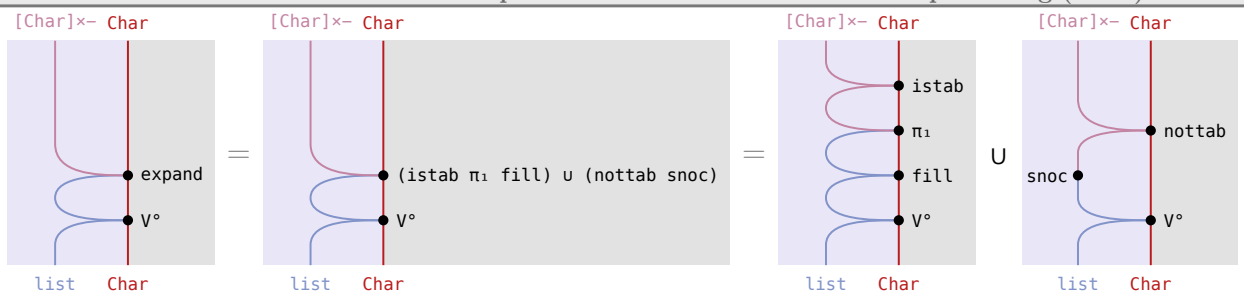
	definition	type	meaning
1	`(|[nil,expand]|) = detab`	`String→ String`	replace every tab by blanks up to the next tab stop; `H=detab°`
2	`expand(xs,a) ≜ if a = tb then fill(xs) else snoc(xs,a)`	`String`	a tab fills the line to the next stop; any other character is appended
3	`fill(xs) ≜ xs#blanks(n−col(xs) % n)`	`String`	append blanks up to the next multiple of `n` columns
4	`col(wrap(a)) = 0, col(snoc(x,a)) = if a = nl then 0 else col(x) + 1`	`String→ℕ`	the column the string ends at: characters since the last newline
5	`xs R ys ⇔ length(xs) ≤ length(ys)`	`[Char]→ [Char]`	`x` is no longer than `y`
6	`a U b ⇔ a = tb ∨ a = b`	`Char→Char`	`a` is the tab or `a=b`
7	`xs V ys ⇔ prefix(xs,ys) ∧ fill(xs) = fill(ys)`	`[Char]→ [Char]`	a prefix that fills to the same string
8	`Q ≜ 𝟙 + (V × U)`	`F[Char]→ F[Char]`	compare two decompositions by `V` on the strings and `U` on the characters
9	`unfill(wrap(a)) = [], unfill(snoc(x,a)) = if a = blank ∧ col(snoc(x,a)) % n ≠ 0 then unfill(x) else snoc(x,a)`	`String→ String`	the shortest prefix that fills to the same string
10	`tbc(wrap(a)) = 0, tbc(snoc(x,a)) = if a = blank ∧ col(snoc(x,a)) % n ≠ 0 then tbc(x) + 1 else 0`	`String→ℕ`	the number of trailing blanks
11	`triple ≜ ⟨unfill entab,⟨tbc,col⟩⟩`	`[Char]→ String×(ℕ×ℕ)`	what `entab` keeps: the unfilled output, its trailing blank count and its column

§10.2.2. Derivation

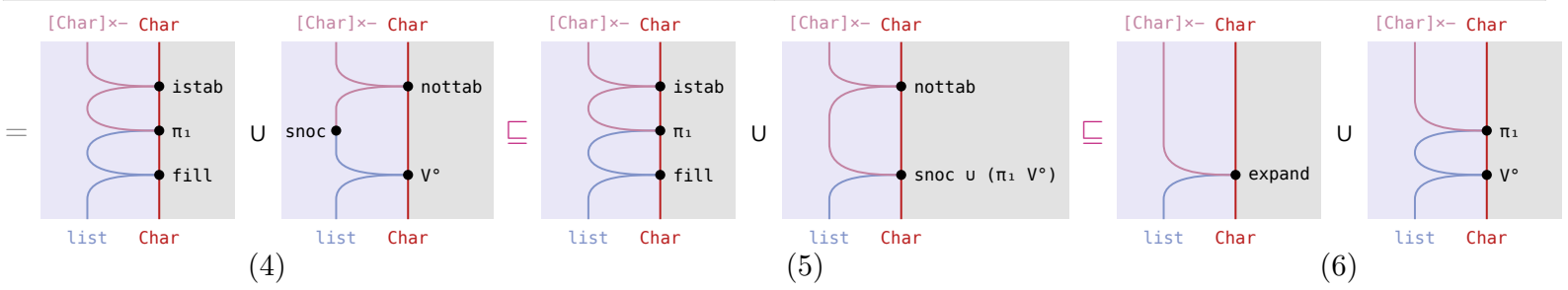
0 < n (10.2.2a)

⇒ expand V° ⊑ expand ∪ (π₁ V°)

shortening the output of one **expand** step to a **V**-smaller string either keeps the whole step (**expand**) or drops its character and shortens its input string (**π₁V°**)



1	expand V°	expand = (istab π₁ fill) ∪ (nottab snoc)
2	= ((istab π₁ fill) ∪ (nottab snoc)) V°	(S ∪ T) R = (S R) ∪ (T R)
3	= (istab π₁ fill V°) ∪ (nottab snoc V°)	0 < n; blank ≠ nl ⇒ fill V° = fill

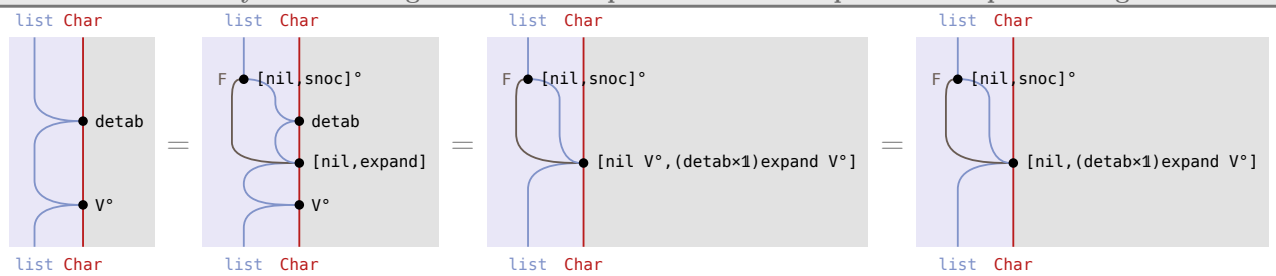


4	= (istab π₁ fill) ∪ (nottab snoc V°)	0 < n; blank ≠ nl ⇒ snoc V° ⊑ snoc ∪ (π₁ V°)
5	⊑ (istab π₁ fill) ∪ (nottab (snoc ∪ (π₁ V°)))	(istab π₁ fill) ∪ (nottab (snoc ∪ X)) ⊑ expand ∪ X
6	⊑ expand ∪ (π₁ V°)	

`0 < n` (10.2.2b)

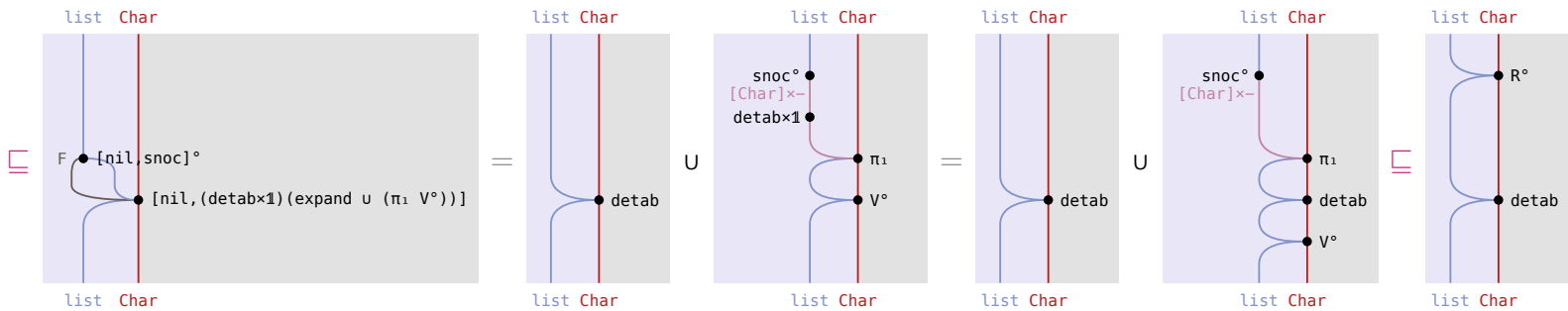
`⇒ detab V° ⊑ R° detab`

any **V**-shortening of **detab**'s output is **detab**'s output on an input no longer than the given one



(1) (2) (3) (4)

1	`detab V°`	`detab = [nil,snoc]° F(detab) [nil,expand]`
2	`= [nil,snoc]° F(detab) [nil,expand] V°`	`[R,expand] Z = [R Z,expand Z]`
3	`= [nil,snoc]° F(detab) [nil V°,expand V°]`	`nil V° = nil`
4	`= [nil,snoc]° F(detab) [nil,expand V°]`	(10.2.2a) `0 < n` `⇒ expand V° ⊑ expand ∪ (π₁ V°)`



(5) (6) (7) (8)

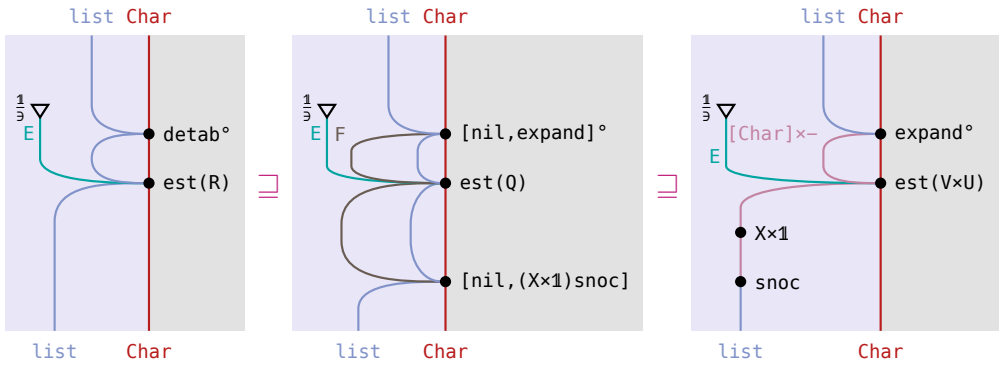
5	`⊑ [nil,snoc]° F(detab) [nil,expand ∪ (π₁ V°)]`	`[nil,snoc]° F(detab) [nil,expand ∪ X] = detab ∪ (snoc° (detab × 𝟙) X)`
6	`= detab ∪ (snoc° (detab × 𝟙) π₁ V°)`	`snoc° (detab × 𝟙) π₁ Y = snoc° π₁ detab Y`
7	`= detab ∪ (snoc° π₁ detab V°)`	`0 < n` `⇒ detab ∪ (snoc° π₁ detab V°) ⊑ R° detab`
8	`⊑ R° detab`	

`0 < n` (10.2.2c)

`⇒ (μX : [nil,expand]° ∋ est(Q) F(X) [nil,snoc]) ⊑ detab° ∋ est(R)`

the shortest input **detab** expands to the given output is one pass along that output, holding each blank back and cashing the held blanks in for a tab wherever the column reaches a tab stop

the specification — `detab(wrap(a)) = []`, `detab(snoc(x,a)) = expand(detab(x),a)` , `⦇[nil,expand]⦈ = detab` ; `detab entab=𝟙` and nothing shorter does

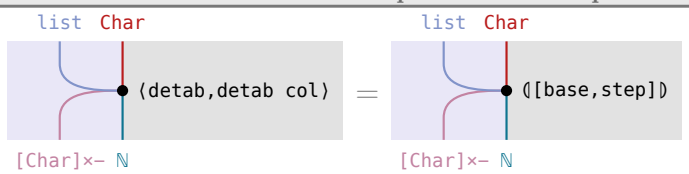


(1) (2) (3)

1	`detab°/∋ est(R)`	at `Q≜𝟙+(V×U)` : one character of input is decided at each step. `F(⊤,R)α⊑αR`. `detab prefix⊑R° detab` is FALSE , — at `n=8`, `detab [a,b,c,d,e,TB]=[a,b,c,d,e,BL,BL,BL]`, whose prefix `[a,b,c,d,e,BL,BL]` is longer than any input giving it, and `detab V°⊑R° detab` holds. `expand V°⊑expand ∪ (π₁V°)` — shortening the output either leaves the last step alone or discards it
2	`⊒ [nil,expand]°/∋ est(Q) F(X) [nil,snoc]`	Proposition 10.1: `nil` and `expand` have disjoint ranges. The greedy step is to emit a tab whenever a tab is legal, consuming all the blanks back to the previous tab stop
3	`⊒ expand°/∋ est(V × U) (X × 𝟙) snoc`	

⟨detab,detab col⟩ = ⦇[base,step]⦈ (10.2.2d)

one pass over the input carries the output so far together with its column



(1) (2)

1	`⟨detab,detab col⟩`	both components are folds over the same input; `step` on `(detab x,col(detab x))` is `(detab,col detab)` one character further — not a tabulated row
2	`= ⦇[base,step]⦈`	

Exercise 10.1

- `⦇[base,step]⦈(convert(xs),r) ⇔ r = loop(step)(([],0),xs)`
  the fold of `[base,step]` over the input converted to a snoc-list is the left loop of `step` from `base`
- `π₁(loop(step)(([],c₀),xs)) = loop'((f,g),(c₀,xs))`
  Exercise 10.1: when `step` only appends `f(c,a)` to the output and moves the state by `g`, the output of the loop is `loop'(f,g)`, which never carries the output

(10.2.2e)

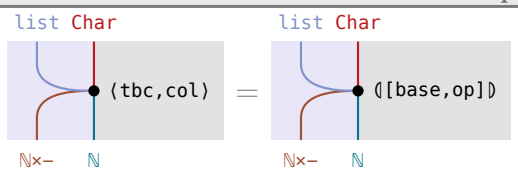
- `a ≠ tb ⇒ expand(ys,b) = snoc(xs,a) ⇔ fill(ys) = snoc(xs,a) ∧ b = tb ∨ (ys,b) = (xs,a)`
  `expand` produces `x#[a]` from a tab after a string that fills to it, or from `x` and `a`
- `0 < n ⇒ (∃ys. fill(ys) = snoc(xs,a)) ⇔ a = blank ∧ col(snoc(xs,a)) % n = 0`
  a string is a `fill` exactly when it ends in a blank on a tab stop

(10.1)

- `entab(xs) = entab(unfill(xs))#blanks(tbc(xs))`
  (10.1): the output of `entab` is that of `unfill x` followed by the trailing blanks held back
- `tbc[] = 0`
  the empty string has no trailing blanks
- `tbc(snoc(xs,a)) = if a = blank ∧ col(snoc(xs,a)) % n ≠ 0 then tbc(xs) + 1 else 0`
  a blank off a tab stop adds one held blank; anything else releases them all

⟨tbc,col⟩ = ⦇[base,op]⦈ (10.2.2f)

one pass counts the held blanks and the column together

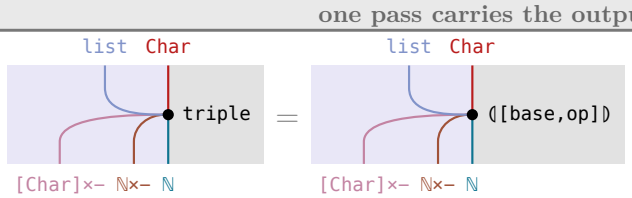


(1) (2)

1	`⟨tbc,col⟩`	`op` on `(tbc x,col x)` is `(tbc,col)` one character further, from the equations for `tbc` — (10.1) `entab(xs) = entab(unfill(xs))#blanks(tbc(xs))`, `tbc[] = 0`, `tbc(snoc(xs,a)) = if a = blank ∧ col(snoc(xs,a)) % n ≠ 0 then tbc(xs) + 1 else 0`
2	`= ⦇[base,op]⦈`	

triple = ⦇[base,op]⦈ (10.2.2g)

one pass carries the output up to the held blanks, their count and the column

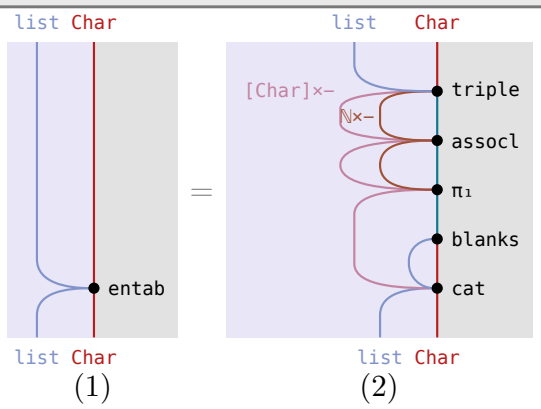


(1) (2)

1	`triple`	`op` on `triple x` is `triple` one character further: (10.1) where the blanks are released — (10.1) `entab(xs) = entab(unfill(xs))#blanks(tbc(xs))`, `tbc[] = 0`, `tbc(snoc(xs,a)) = if a = blank ∧ col(snoc(xs,a)) % n ≠ 0 then tbc(xs) + 1 else 0`
2	`= ⦇[base,op]⦈`	

entab = triple assocl π₁ (𝟙 × blanks) cat (10.2.2h)

entab is **triple** with the held blanks appended to its output



1	`entab`	(10.1) — (10.1) `entab(xs) = entab(unfill(xs)) #blanks(tbc(xs)) , tbc[] = 0 , tbc(snoc(xs,a)) = if a = blank ∧ col(snoc(xs,a)) % n ≠ 0 then tbc(xs) + 1 else 0`
2	`= triple assocl π₁ (𝟙 × blanks) cat`	

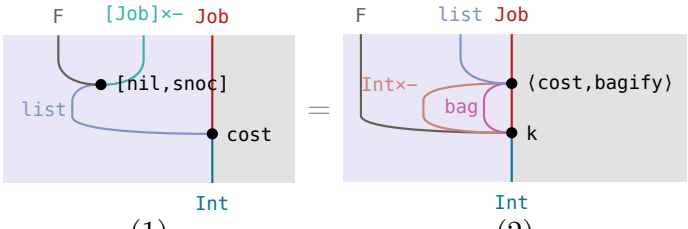
§10.3. The minimum tardiness problem

(10.3a)

	definition	type	meaning
1	`bag(Job) ≜ Quotient(perm)`	`𝐴`	a bag of jobs: a list up to reordering
2	`snag(p) ≜ π₂(p) :: π₁(p)`	`bag(Job)`	put one job into a bag
3	`(|[nil,snag]|) = bagify`	`[Job]→bag(Job)`	forget the order of a schedule; `H=bagify°` lists every schedule of a bag
4	`penalty(xs,j) ≜ (sum(ct′)(xs) + ct(j)−dt(j))wt(j)`	`Int`	how late job `j` finishes after the schedule `xs`, times its weight
5	`cost(wrap(a)) = 0, cost(snoc(x,a)) = bmax(cost(x),penalty(x,a))`	`[Job]→Int`	the largest penalty of any job in the schedule
6	`bmax(a,b) ≜ if a ≤ b then b else a`	`Int`	the larger of two numbers
7	`R ≜ cost ≤ cost°`	`[Job]→[Job]`	`xs` costs no more than `ys`
8	`p add ys ⇔ add(π₂(p),ys)`	`[Job]×Job→[Job]`	insert `j` anywhere in `xs`; `perm≜bagify` `bagify°=(|[nil,add]|)`
9	`k ≜ [nil,assocr (𝟙×((bagify°×𝟙) penalty)) bmax]`	`F(Int×bag(Job))→ Int`	the cost of a schedule from the cost of its front and the penalty of its last job
10	`f(l(val)) = 0, f(r(p)) = (bagify°×𝟙) penalty(p)`	`F(bag(Job))→Int`	the penalty of the last job, read off the bag before it
11	`u Q v ⇔ f(u) ≤ f(v)`	`F(bag(Job))→ F(bag(Job))`	the last job of `u` gets no larger penalty than that of `v`
12	`p Q' q ⇔ (bagify°×𝟙) penalty(p) ≤ (bagify°×𝟙) penalty(q)`	`bag(Job)×Job→ bag(Job)×Job`	`Q` on the `snag` summand: compare the penalties of the two last jobs
13	`g ≜ [zero,penalty]`	`F[Job]→Int`	(10.5): the penalty of the last job, zero for the empty schedule
14	`m ≜ [zero,π₁ cost]`	`F[Job]→Int`	(10.6), B&dM's `h` renamed as in `(∀j. 0 ≤ ct(j)); (∀j. 0 ≤ wt(j)) ⇒ (μX : [nil,snag]° ∋ est(Q) F(X) [nil,snoc]) ⊑ bagify° ∋ est(R)` : the cost of the schedule before the last job

`[nil,snoc] cost = F((cost,bagify)) k` (10.3b)

the cost of a schedule is got from the cost and the bag of the schedule before its last job, and that job

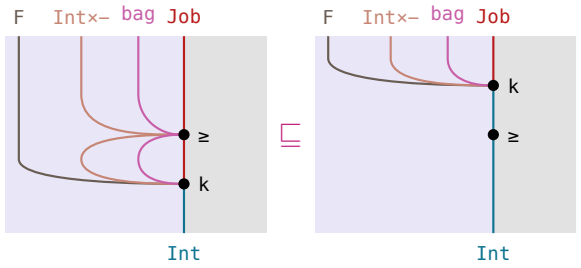


(1) (2)

1	`[nil,snoc] cost`	`cost (xs#[j])=bmax (cost xs,penalty (xs,j))`, and `penalty(xs,j)` reads only the bag of `xs` — `penalty(xs,j) ≜ (sum(ct′)(xs) + ct(j)−dt(j))wt(j)`
2	`= F((cost,bagify)) k`	

F(≥ × 𝟙) k ⊑ k ≥ — Exercise 10.5

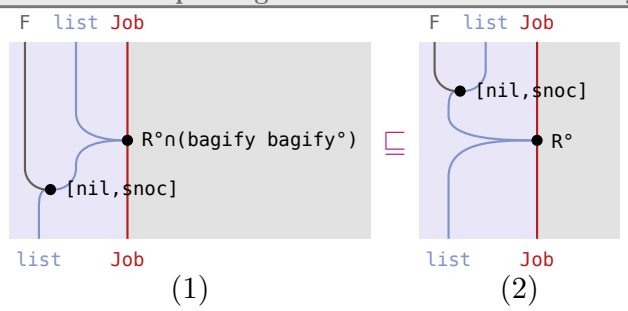
a larger cost before the last job gives a larger cost after it



1	`F(≥ × 𝟙) k`	`bmax` is monotone in its first argument
2	`⊑ k ≥`	

F(R° ∩ (bagify bagify°)) [nil,snoc] ⊑ [nil,snoc] R° — (10.2)

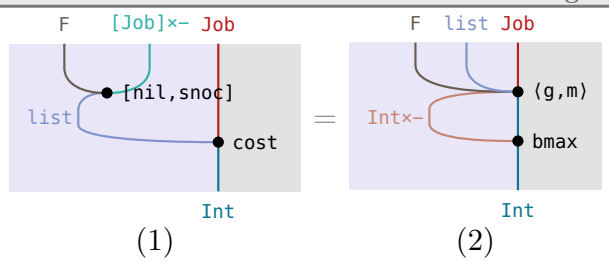
improving the schedule before the last job, among schedules of the same bag, improves the whole schedule



1	`F(R° ∩ (bagify bagify°)) [nil,snoc]`	Proposition 9.3 at `S≜bagify`, `⩽≜≥` — Proposition 9.3 `Map(cost); ≤ = cost ⩽ cost°; Simple(H°); h cost = F(⟨cost,H°⟩) k; F(⩽ × 𝟙) k ⊑ k ⩽ ⇒ F(≤ ∩ (H° H)) h ⊑ h ≤`, with `[nil,snoc] cost = F(⟨cost,bagify⟩) k` and Exercise 10.5 `F(≥ × 𝟙) k ⊑ k ≥`
2	`⊑ [nil,snoc] R°`	

[nil,snoc] cost = ⟨g,m⟩ bmax — (10.4)

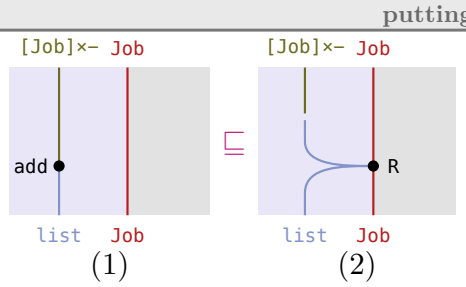
the cost of a schedule is the larger of the penalty of its last job and the cost of the schedule before it



1	`[nil,snoc] cost`	`cost (xs#[j])=bmax (cost xs,penalty (xs,j))` and `bmax` commutes — `cost(wrap(a)) = 0, cost(snoc(x,a)) = bmax(cost(x),penalty(x,a))`
2	`= ⟨g,m⟩ bmax`	

(∀j. 0 ≤ ct(j)); (∀j. 0 ≤ wt(j))
⇒ add ⊑ π₁ R — (10.7)

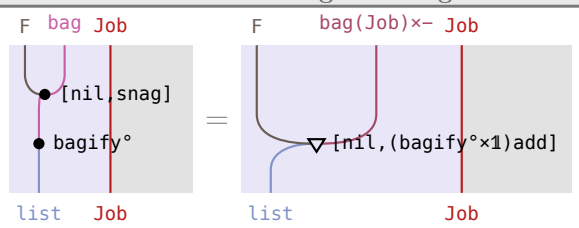
putting one more job anywhere into a schedule never lowers its cost



1	`add`	induction on where `j` goes: every job after it starts later, so no penalty falls
2	`⊑ π₁ R`	

[nil,snag] bagify° = [nil,(bagify° × 𝟙) add] (10.8)

the orderings of a bag with one more job are the orderings of the bag with that job put in anywhere

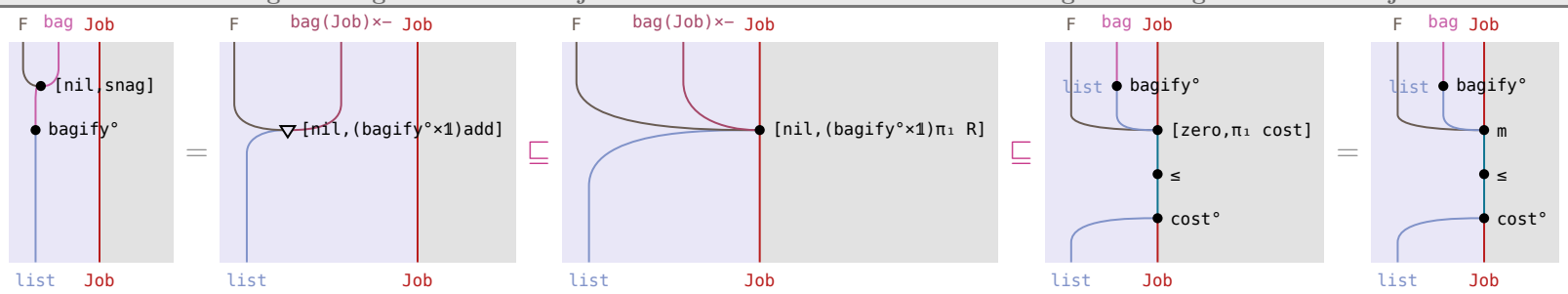


1	[nil,snag] bagify°	take out the last copy of j and put it back — p add ys ⇔ add(π₂(p),ys)
2	= [nil,(bagify° × 𝟙) add]	

(∀j. 0 ≤ ct(j)); (∀j. 0 ≤ wt(j)) (10.3c)

⇒ [nil,snag] bagify° ⊑ F(bagify°) m ≤ cost°

an ordering of a bag with one more job costs at least the cost of the ordering of the bag before its last job

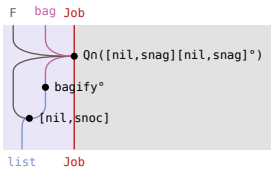


1	[nil,snag] bagify°	(10.8) [nil,snag] bagify° = [nil,(bagify° × 𝟙) add]
2	= [nil,(bagify° × 𝟙) add]	(10.7) (∀j. 0 ≤ ct(j)); (∀j. 0 ≤ wt(j)) ⇒ add ⊑ π₁ R
3	⊑ [nil,(bagify° × 𝟙) π₁ R]	[nil,π₁ R] ⊑ [zero,π₁ cost] ≤ cost°
4	⊑ F(bagify°) [zero,π₁ cost] ≤ cost°	
5	= F(bagify°) m ≤ cost°	

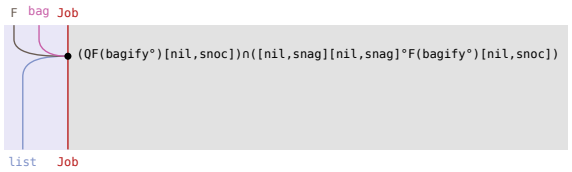
(∀j. 0 ≤ ct(j)); (∀j. 0 ≤ wt(j)) **(10.3)**

⇒ (Q ∩ ([nil,snag] [nil,snag]°)) F(bagify°) [nil,snoc] ⊑ F(bagify°) [nil,snoc] R

a schedule of a bag ending in a job of least penalty is no worse than any schedule of the same bag



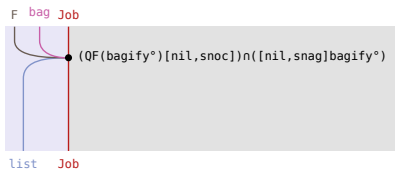
⊑



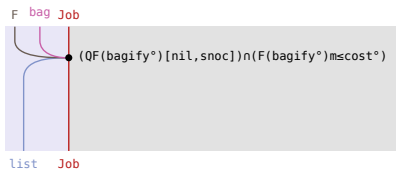
(1) (2)

1	(Q ∩ ([nil,snag] [nil,snag]°)) F(bagify°) [nil,snoc]	(Q ∩ T) R ⊑ (Q R) ∩ (T R)
2	⊑ (Q F(bagify°) [nil,snoc]) ∩ ([nil,snag] [nil,snag]° F(bagify°) [nil,snoc])	[nil,snag]° F(bagify°) [nil,snoc] = bagify°

=



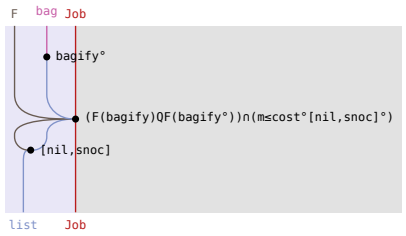
⊑



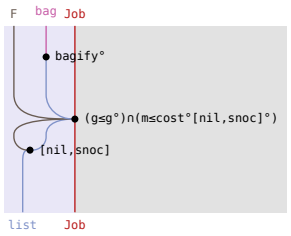
(3) (4)

3	= (Q F(bagify°) [nil,snoc]) ∩ ([nil,snag] bagify°)	(10.3c) (∀j. 0 ≤ ct(j)); (∀j. 0 ≤ wt(j)) ⇒ [nil,snag] bagify° ⊑ F(bagify°) m ≤ cost°
4	⊑ (Q F(bagify°) [nil,snoc]) ∩ (F(bagify°) m ≤ cost°)	(Q F(bagify°) [nil,snoc]) ∩ (F(bagify°) M) ⊑ F(bagify°) ((F(bagify) Q F(bagify°)) ∩ (M [nil,snoc]°)) [nil,snoc]

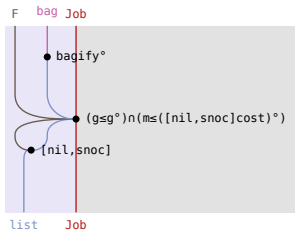
⊑



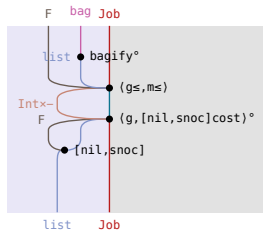
=



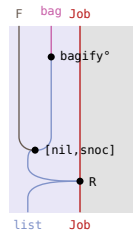
=



=



⊑

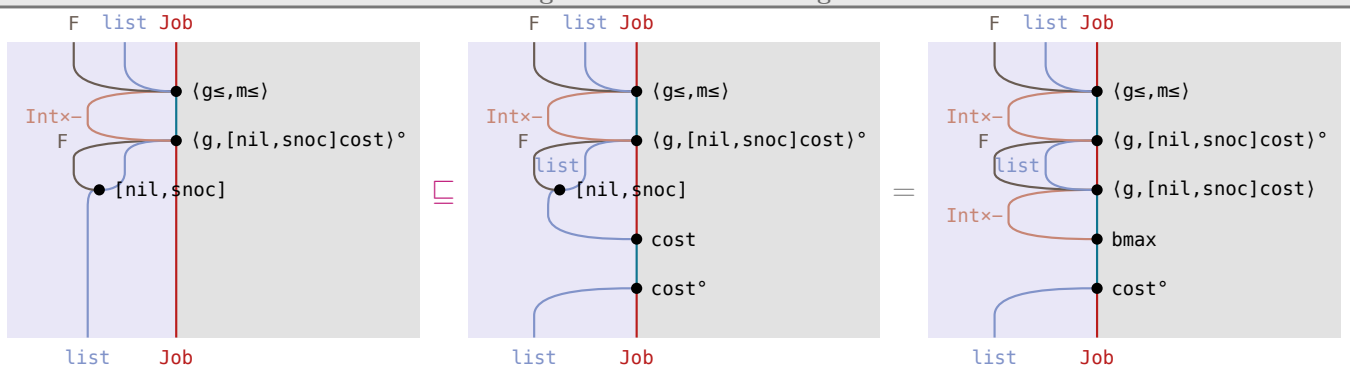


(5) (6) (7) (8) (9)

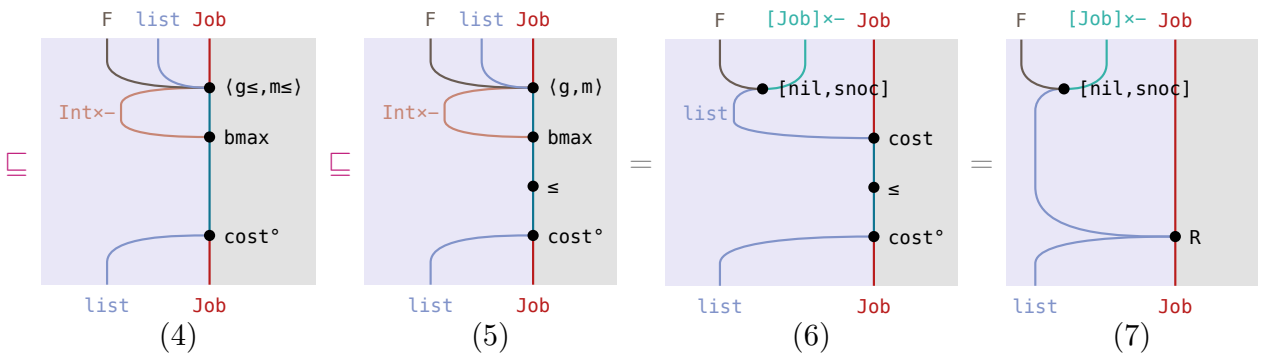
5	⊑ F(bagify°) ((F(bagify) Q F(bagify°)) ∩ (m ≤ cost° [nil,snoc]°)) [nil,snoc]	F(bagify) Q F(bagify°) = g ≤ g°
6	= F(bagify°) ((g ≤ g°) ∩ (m ≤ cost° [nil,snoc]°)) [nil,snoc]	(con cost)° = cost° con°
7	= F(bagify°) ((g ≤ g°) ∩ (m ≤ ([nil,snoc] cost)°)) [nil,snoc]	⟨X,Y⟩ ⟨g,S⟩° = ⟨X g°⟩ ∩ ⟨Y S°⟩
8	= F(bagify°) ⟨g ≤,m ≤⟩ ⟨g,[nil,snoc] cost⟩° [nil,snoc]	(10.3d) ⟨g ≤,m ≤⟩ ⟨g,[nil,snoc] cost⟩° [nil,snoc] ⊑ [nil,snoc] R
9	⊑ F(bagify°) [nil,snoc] R	

⟨g ≤,m ≤⟩ ⟨g,[nil,snoc] cost⟩° [nil,snoc] ⊑ [nil,snoc] R (10.3d)

a job whose penalty is at most the last penalty, put after a schedule costing at most the cost before the last job, gives a schedule costing at most the whole



1	⟨g ≤,m ≤⟩ ⟨g,[nil,snoc] cost⟩° [nil,snoc]	𝟙 ⊑ cost cost°
2	⊑ ⟨g ≤,m ≤⟩ ⟨g,[nil,snoc] cost⟩° [nil,snoc] cost cost°	[nil,snoc] cost = ⟨g,[nil,snoc] cost⟩ bmax
3	= ⟨g ≤,m ≤⟩ ⟨g,[nil,snoc] cost⟩° ⟨g,[nil,snoc] cost⟩ bmax cost°	⟨g,[nil,snoc] cost⟩° ⟨g,[nil,snoc] cost⟩ ⊑ 𝟙

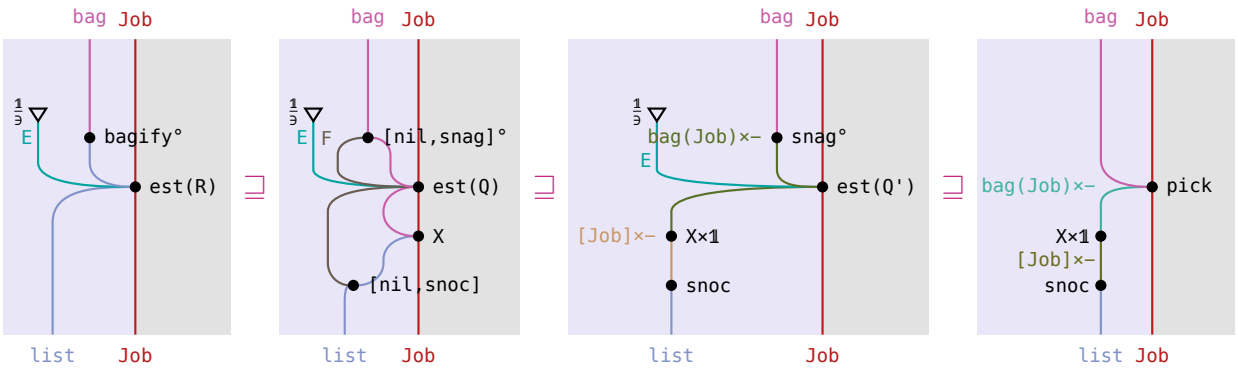


4	⊑ ⟨g ≤,m ≤⟩ bmax cost°	⟨g ≤,m ≤⟩ bmax ⊑ ⟨g,m⟩ bmax ≤
5	⊑ ⟨g,m⟩ bmax ≤ cost°	(10.4) [nil,snoc] cost = ⟨g,m⟩ bmax
6	= [nil,snoc] cost ≤ cost°	R = cost ≤ cost°
7	= [nil,snoc] R	

schedule=(null→nil,pick (schedule×𝟙) snoc), (∀j. 0 ≤ ct(j)); (∀j. 0 ≤ wt(j)) (10.3e)
⇒ (μX : $\frac{[nil,snag]^\circ}{\ni}$ est(Q) F(X) [nil,snoc]) ⊑ $\frac{bagify^\circ}{\ni}$ est(R)

an ordering of the given bag with least maximum penalty is got by taking a job of least penalty out of the bag, putting it last, and scheduling what is left the same way

the specification — ⦇[nil,snag]⦈ = bagify



1	$\frac{bagify^\circ}{\ni}$ est(R)	No greedy **reduce** exists — one would also solve every prefix of the input, and the best schedule of a prefix need not extend to a best schedule of the whole
2	⊒ $\frac{[nil,snag]^\circ}{\ni}$ est(Q) F(X) [nil,snoc]	Proposition 10.1: nil and snag have disjoint ranges
3	⊒ $\frac{snag^\circ}{\ni}$ est(Q') (X × 𝟙) snoc	pick⊑$\frac{snag^\circ}{\ni}$ est(Q'), a partial function, quadratic in the number of jobs
4	⊒ pick (X × 𝟙) snoc	

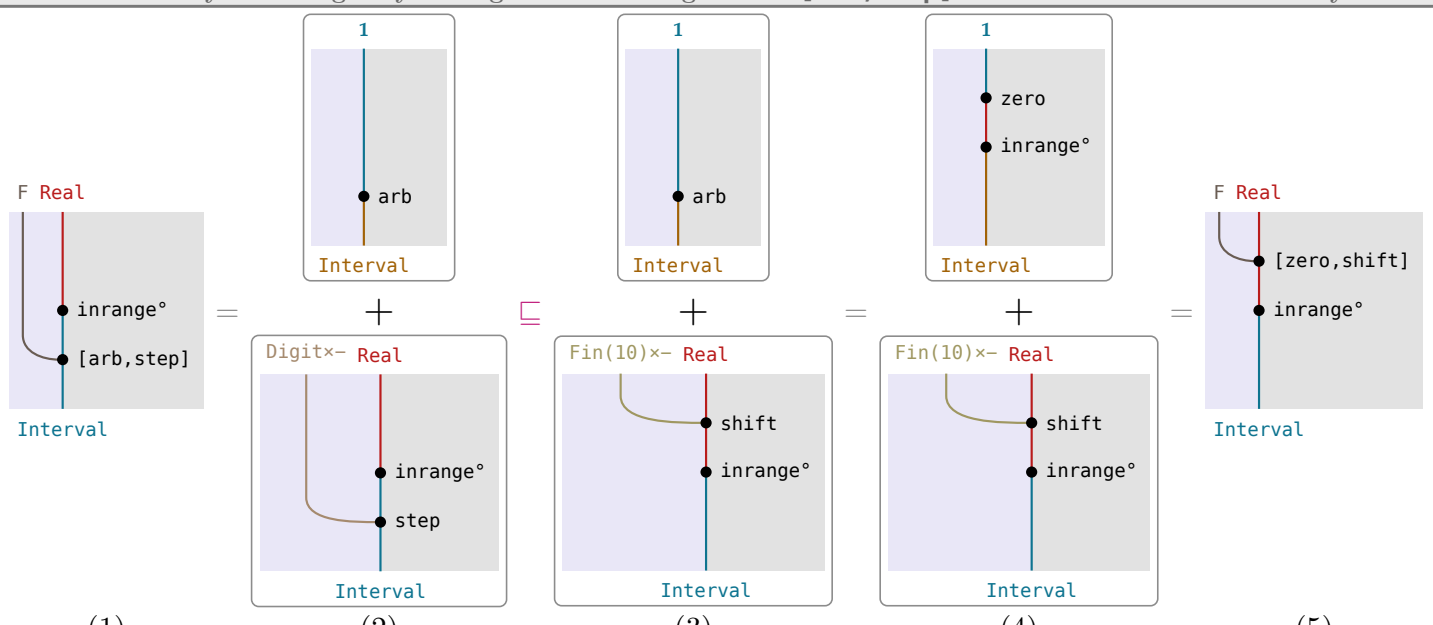
§10.4. The TeX problem

(10.4a)

	definition	type	meaning
1	`intern ≜ val round`	`[Digit]→[0,2¹⁶)`	the value of a decimal fraction, rounded to `16` binary places
2	`val ≜ (|[zero,shift]|)`	`[Digit]→Real`	the real number a digit list denotes after the decimal point
3	`shift(d,r) ≜ (d + r)/10`	`Digit×Real→Real`	put digit `d` in front of `r`: `(d+r)/10`
4	`round° = interval inrange`	`Real→[0,2¹⁶)`	the nearest integer to `2¹⁶r`: `round r=n` iff `2n−1<2¹⁷r<2n+1`
5	`interval(n) ≜ ((2n−1)/w,(2n + 1)/w)`	`[0,2¹⁶)→ Interval`	the open interval `((2n−1)/2¹⁷, (2n+1)/2¹⁷)` of the reals that round to `n`
6	`p inrange r ⇔ lo(p) < r ∧ r < hi(p)`	`Interval→Real`	`r` lies strictly inside `(a,b)`
7	`Legal(p) ≜ 0 < hi(p) ∧ hi(p) < 1 ∧ lo(p) < hi(p)`	`Prop`	(10.9): `0<b<1` and `a<b`; `Interval` is the legal pairs
8	`x arb p ⇔ lo(p) < 0 ∧ 0 < hi(p)`	`𝟙→Interval`	any legal interval
9	`step(p) ≜ (shift(π₁(p),lo(π₂(p))),shift(π₁(p),hi(π₂(p))))`	`Digit×Interval→ Interval`	put digit `d` in front of both ends: `((d+a)/10,(d+b)/10)`
10	`H ≜ (|[arb,step]|)°`	`Interval→ [Digit]`	every digit list whose interval the given one is
11	`xs R ys ⇔ length(xs) ≤ length(ys)`	`[Digit]→[Digit]`	`x` is no longer than `y`
12	`l ≜ inl`	`𝟙→FX`	the injection of the empty case into `FX=1+(Digit×X)`
13	`r ≜ inr`	`Digit×X→FX`	the injection of the digit case into `FX=1+(Digit×X)`
14	`! ≜ ⊸ ()`	`Digit×Interval→ 𝟙`	the one map to `𝟙`
15	`Q ≜ (l° !° r) ∪ 𝟙`	`F(Interval)→ F(Interval)`	`u` is the empty case and `z` a digit case, or `u=z`
16	`w ≜ 131072`	`Int`	`2¹⁷`

`(|[arb,step]|) ⊑ val inrange°` (10.4b)

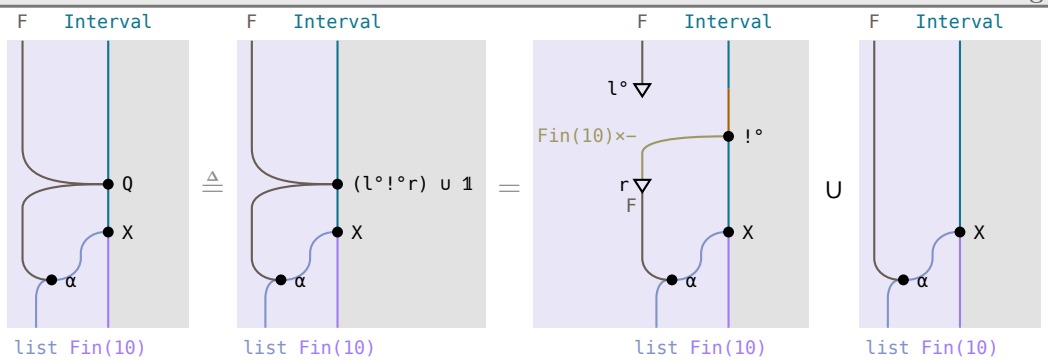
every interval got by folding a decimal's digits with `[arb,step]` has the decimal's value strictly inside it



1	`F(inrange°) [arb,step]`	`F(S) [arb,step] = [arb,(𝟙 × S) step]`
2	`= [arb,(𝟙 × inrange°) step]`	`(𝟙 × inrange°) step ⊑ shift inrange°`
3	`⊑ [arb,shift inrange°]`	`zero inrange° = arb`
4	`= [zero inrange°,shift inrange°]`	`[zero,shift] Z = [zero Z,shift Z]`
5	`= [zero,shift] inrange°`	

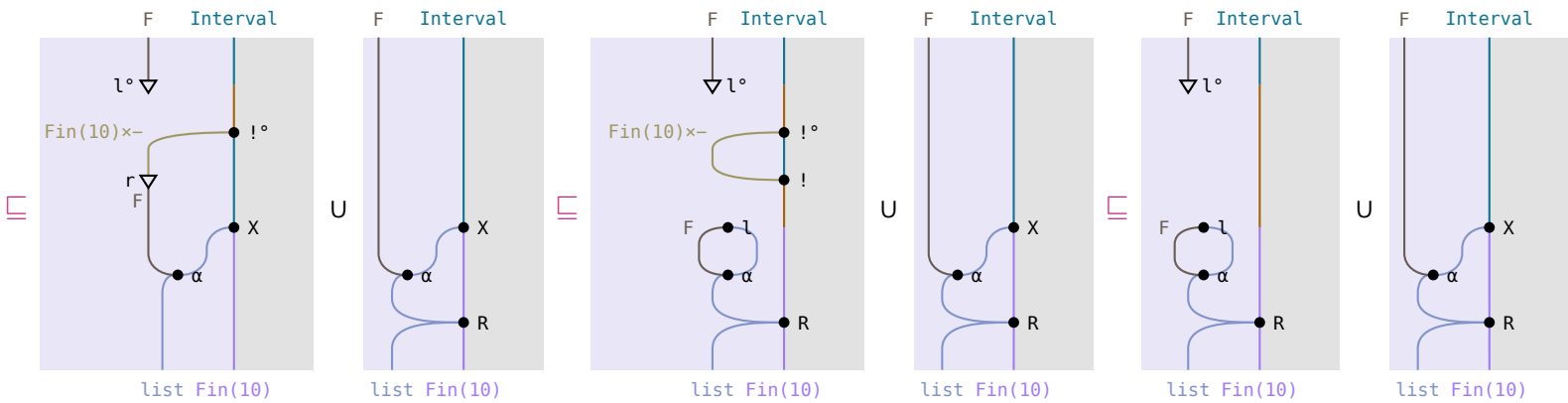
`Q F(X) α ⊑ F(X) α R` (10.4c)

choosing by `Q` among the one-step decompositions before building with `F(X)` and `α` yields only decimals that are `R`-related to one built without choosing



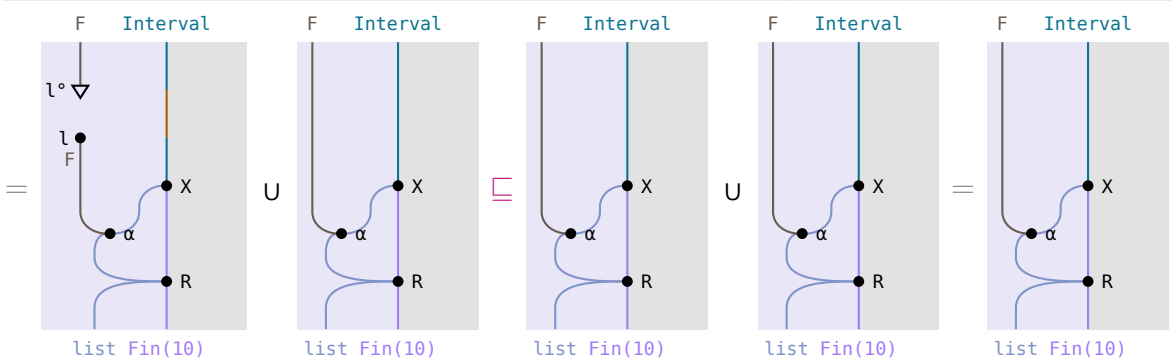
(1) (2) (3)

1	`Q F(X) α`	`Q ≜ (l° !° r) ∪ 𝟙`
2	`≜ ((l° !° r) ∪ 𝟙) F(X) α`	`(S ∪ T) R = (S R) ∪ (T R)`
3	`= (l° !° r F(X) α) ∪ (F(X) α)`	`𝟙 ⊑ R`



(4) (5) (6)

4	`⊑ (l° !° r F(X) α) ∪ (F(X) α R)`	`r F(X) α ⊑ ! l α R`
5	`⊑ (l° !° ! l α R) ∪ (F(X) α R)`	`!° ! ⊑ 𝟙`
6	`⊑ (l° l α R) ∪ (F(X) α R)`	`l F(X) = l`

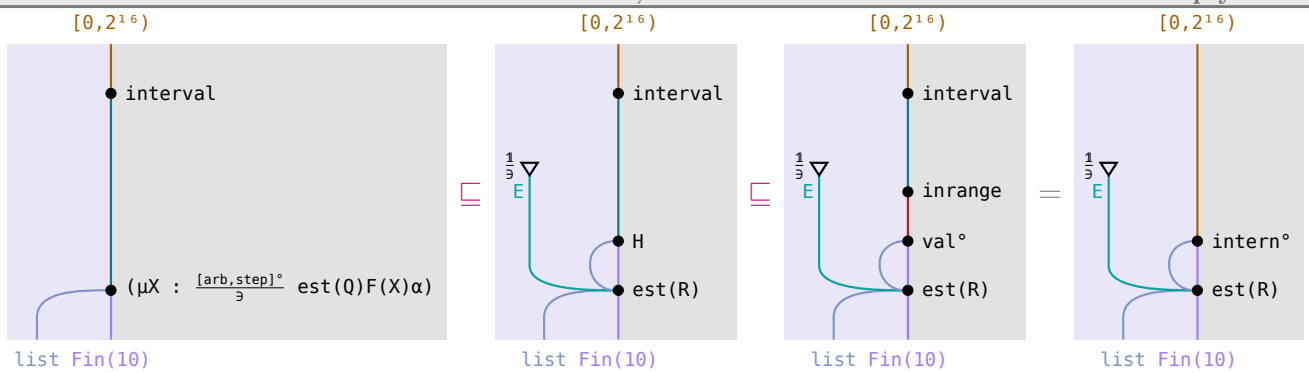


(7) (8) (9)

7	`= (l° l F(X) α R) ∪ (F(X) α R)`	`l° l ⊑ 𝟙`
8	`⊑ (F(X) α R) ∪ (F(X) α R)`	`R ∪ R = R`
9	`= F(X) α R`	

extern(n)=f(2n−1,2n+1), interval (μX : $\frac{[arb,step]^\circ}{\ni}$ est(Q) F(X) α) ⊑ $\frac{intern^\circ}{\ni}$ est(R) (10.4d)

a shortest decimal whose internal representation is the given multiple of 2^{-16} is got by emitting the one digit the interval of admissible reals allows, until that interval contains zero and the empty decimal will do



1	interval (μX : $\frac{[arb,step]^\circ}{\ni}$ est(Q) F(X) α)	$\frac{[arb,step]^\circ}{\ni}$ est(Q) F($\frac{H}{\ni}$ est(R)) α ⊑ $\frac{H}{\ni}$ est(R)
2	⊑ interval $\frac{H}{\ni}$ est(R)	$\frac{H}{\ni}$ est(R) ⊑ $\frac{inrange\ val^\circ}{\ni}$ est(R)
3	⊑ interval $\frac{inrange\ val^\circ}{\ni}$ est(R)	$\frac{intern^\circ}{\ni}$ = interval $\frac{inrange\ val^\circ}{\ni}$
4	= $\frac{intern^\circ}{\ni}$ est(R)	

(10.4e)

- `f(a,b) = xs ⇔ a < 0 ∧ xs = [] ∨ ¬a < 0 ∧ (∃d. ∃ys. d = ⌊10b⌋ ∧ f(10a−d,10b−d) = ys ∧ xs = [d]⧺ys)`
  `f` gives `(a,b)` the empty decimal when `a<0`, and otherwise the digit `d=⌊10b⌋` followed by what it gives `(10a−d,10b−d)`
- `(μX : [arb,step]° / ∋ est(Q) F(X) α) = f`
  the least solution of the greedy recursion of `interval (μX : [arb,step]° / ∋ est(Q) F(X) α) ⊑ intern° / ∋ est(R)` is `f`
- `d = ⌊10b⌋; e = ⌊10b⌋ ⇒ d = e`
  Exercise 10.12: two digits `d` and `e` with `0<10b−d<1` and `0<10b−e<1` are equal
- `0 < q−p ∧ q < w; p < 0 ⇒ f(p,q) = []`
  the program's first clause: `f(p,q)=[]` when `p<0`
- `0 < q−p ∧ q < w; ¬p < 0 ⇒ f(p,q) = [10q/w]⧺f(10p−w·(10q/w),10q−w·(10q/w))`
  the program's second clause: with `d=(10q) div w`, `f(p,q)` is `d` followed by `f(10p−w·d,10q−w·d)`
- `0 < q−p ∧ q < w; Reach(p,q) ⇒ f(p/w,q/w) = f(p,q)`
  on the pairs the program reaches from `interval n`, the integer `f` at `(p,q)` is a value of the rational `f` at `(p/w,q/w)`
- `interval f = extern`
  `f` after `interval` is the program `extern`, whose value at `n` is `f(2n−1,2n+1)`

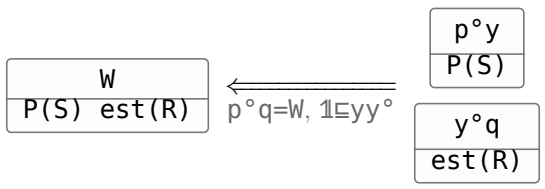
§A. Appendix

§A.1. P(S) est(R)=(∋S)∩(∈\(SR°))

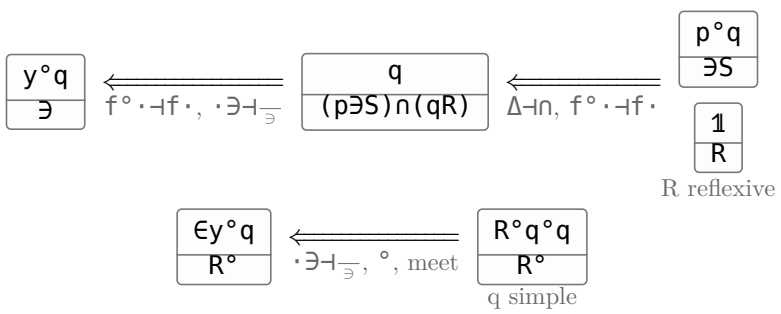
(A.1a)

(p,q) tabulates W≜(∋S)∩(∈\(SR°)), and y≜ $\frac{(p∋S)∩(qR)}{∋}$, a map.

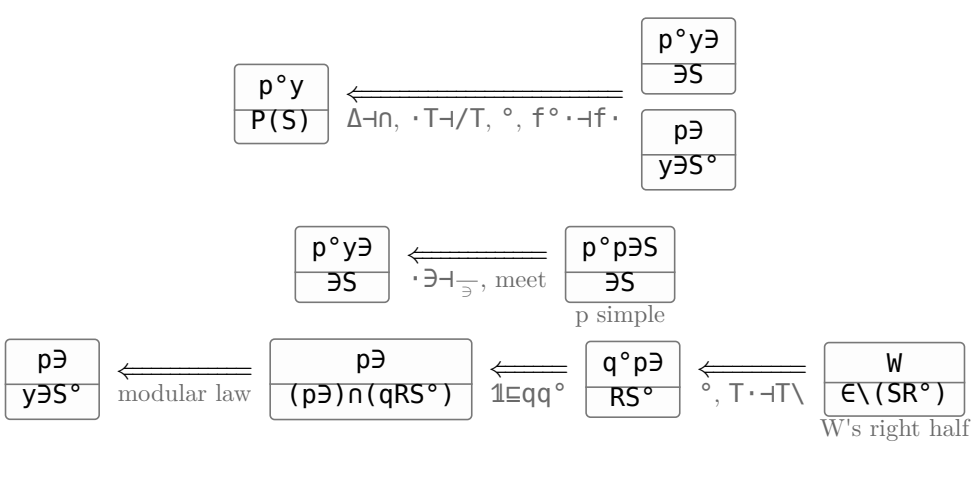
(A.1b)



(A.1c)



(A.1d)



§A.2. P(est(R)) est(R)=P(dom(est(R))) union est(R)

(A.2a)

P(est(R)) est(R) ═══ dom(est(R)) est(R)=est(R) ═══ P(dom(est(R)) est(R)) est(R)

═══ P a relator ═══ P(dom(est(R)))P(est(R)) est(R)

(7.11) $\overline{\sqsubseteq}$ $R^2 \sqsubseteq R$ ⇒ P(est(R)) est(R) ⊑ union est(R)

P(dom(est(R))) union est(R)

(A.2b)

P(dom(est(R))) union est(R) / ∋est(R) ⟸ 𝟙 ⊑ R ⟸ P(dom(est(R))) union est(R) / P(est(R)) est(R)

⇒ P(S) est(R) = (∋ S) ∩ (∈ \ (S R°)) at S:=est(R), Δ⊣∩, T·⊣T\

∈P(dom(est(R))) union est(R) / est(R)R°

P(dom(est(R))) union est(R) / ∋est(R) ⟸ P(dom(est(R)))⊑𝟙 ═══ union est(R) / ∋est(R)

T(U∩V)⊑TU∩TV, ·∋⊣$\overline{\ni}$, (7.3) union (∈ \ R) = ∈ \ (∈ \ R) ═══ (∋∋)∩(∈\(∈\R°)) / ∋est(R) ⟸ modular law ═══ ∋(∋∩(∈(∈\(∈\R°)))) / ∋(∋∩(∈\R°))

counit of T · ⊣T\

∈P(dom(est(R))) union est(R) / est(R)R° ⟸ ∈ lax natural ═══ dom(est(R))∈union est(R) / est(R)R°

dom(est(R))⊑est(R) est(R)° ═══ est(R) est(R)°∈union est(R) / est(R)R° ⟸ est(R)°⊑∈, ∈∈union⊑∈ ═══ est(R)∈est(R) / est(R)R°

UP of est

§A.3. Shortest paths on a cylinder, on lists

(A.3a)

	definition	type	note
1	`F(A,X)=A+A×X`	`𝒜→𝒜`	The base functor: one square of the new column, alone or in front of the path so far.
2	`L=list⁺`	`𝒜→𝒜`	A path is a non-empty list of squares, one per column crossed.
3	`α` the initial algebra	`F(μF)→μF`	`α(5)=[5]` and `α(1,[5])=[1,5]`: a path is started by one square, or extended by one.
4	`N` the `n`-tuple relator	`𝒜→𝒜`	One component per row of a column, so the fold carries `n` answers at once.
5	`R≜sum≤sum°`	`[ℕ]→[ℕ]`	The cost of a path, which the cheapest minimises.
6	`setify`	`A[n]→PA`	`setify(1,2,3,4)={1,2,3,4}` — which row a component came from is forgotten.
7	`moves`	`A[n]→P(A[n])`	`moves(x)={up(x),x,down(x)}` — rotated up, unrotated, rotated down.
8	`trans`	`P(A[n])→(P A)[n]`	`trans{(a,b,c),(x,y,z)}=({a,x},{b,y},{c,z})` — component `k` of the result is the set of the `k`-th components.
9	`zip`	`F(a[n])→(F a)[n]`	`zip((1,2,3,4),({[5]},{[6]},{[7]},{[8]}))=((1,{[5]}),(2,{[6]}),(3,{[7]}),(4,{[8]}))`.
10	`cp≜` $\frac{\texttt{F(1,∋)}}{\texttt{∋}}$	`F(PA)→PFA`	`cp(1,{[5],[6],[8]})={(1,[5]),(1,[6]),(1,[8])}`.
11	`gen ≜ F(𝟙,moves trans N(union))` `zip N(cp(F(A,−)) E(α))`	`F(NA,N(P(TA)))→` `N(P(TA))`	`gen((1,2,3,4),({[5]},{[6]},{[7]},{[8]}))` is worked out in `gen ≜ F(𝟙,moves trans N(union))` `zip N(cp(F(A,−)) E(α))` .
12	`paths ≜ (gen) setify union`	`T(NA)→P(TA)`	`paths[(1,2,3,4),(5,6,7,8)]` is the union of `gen ≜ F(𝟙,moves trans N(union)) zip N(cp(F(A,−)) E(α))` ‘s four sets: 12 paths, 3 from each entry row.
13	the specification `paths est(R)`	`[ℕ[n]]→[ℕ]`	A cheapest path from the entry side to the exit side.

§A.3.1. N(E(L A))

(A.3.1a)

```
1  5
2  6
3  7
4  8
```

```
[(1,2,3,4),(5,6,7,8)] : L N Nat
```

(A.3.1b)

```
gen ≜ F(𝟙,moves trans N(union)) zip N(cp(F(A,−)) E(α))

(gen)[(5,6,7,8)] = ({[5]},{[6]},{[7]},{[8]})

gen((1,2,3,4),({[5]},{[6]},{[7]},{[8]})) =
    ( {[1,5],[1,6],[1,8]},
      {[2,5],[2,6],[2,7]},
      {[3,6],[3,7],[3,8]},
      {[4,5],[4,7],[4,8]} )
```

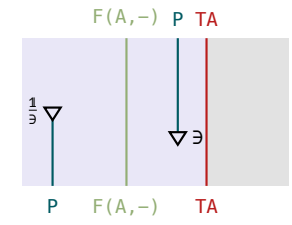
(A.3.1c)

1	`N`	the 4-tuple, one component per row of the entry column
2	`E`	the set of paths that can start in that row
3	`L`	a path is the non-empty list of the squares it crosses

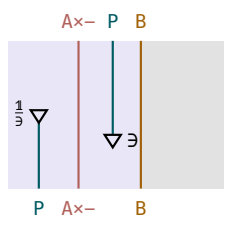
row 1's set has no `[1,7]` — from row 1 only rows 4, 1 and 2 are reachable — and it does have `[1,8]`, because the cylinder glues the bottom row to the top. That is `moves trans N(union)` of `gen` ≜ `F(𝟙,moves trans N(union)) zip N(cp(F(A,−)) E(α))` : component `k` collects the paths of rows `k-1`, `k`, `k+1`.

§A.3.2. The cross product cp≜$\frac{F(\mathbb{1},\ni)}{\ni}$

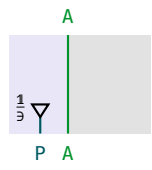
(A.3.2a)



`cp` on all of `F(A,E(L A))`



the `A×−` summand: `∋` picks one path, `𝟙%∋` collects the results



the `A` summand, no `E` to distribute: `𝟙%∋` alone, `a↦{a}`

(A.3.2b)

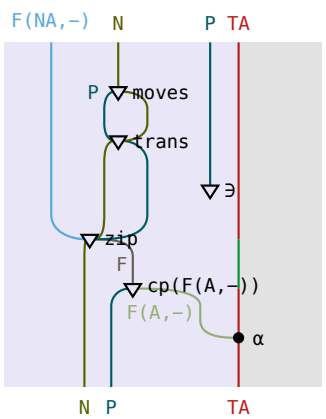
	type	note
1	`F(A,−)=A+A×−`	one wire carries the whole functor, both summands with it; the `∋` under it is `F(𝟙,∋)`
2	`∋:E(L A)→L A`	the `∋` inside `F(𝟙,∋)`: a set of paths, one of them — `L A` is one path, `A` one square
3	`F(𝟙,∋)=𝟙+𝟙×∋`	`:A+A×E(L A)→A+A×L A`, the relator acting on each summand
4	`∋:E(A+A×L A)→A+A×L A`	the `∋` under the bar: a set of cells, one of them — a different `∋`
5	`cp:A+A×E(L A)→E(A+A×L A)`	$\frac{R}{\ni}$ turns `R:X→Y` into `X→E Y`
6	`cp=`$\frac{\mathbb{1}}{\ni}$ `E(F(𝟙,∋))`	what the picture draws: the unit makes the outer `E`, leaving one `∋` — $\frac{R}{\ni}$ = $\frac{\mathbb{1}}{\ni}$ `E(R)`

(A.3.2c)

```
w      = (1,{[5],[6],[8]})            : A×E(L A)     the right summand
𝟙×∋    : A×E(L A) → A×L A                            𝟙 keeps the square 1, ∋ picks one path
         w ↦ (1,[5]), (1,[6]), (1,[8]) : A×L A        one output per path in w
cp(w)  = {(1,[5]),(1,[6]),(1,[8])}    : E(A+A×L A)   the three of them, collected
```

§A.3.3. gen=F(𝟙,moves trans N(union)) zip N(cp P(α))

(A.3.3a)



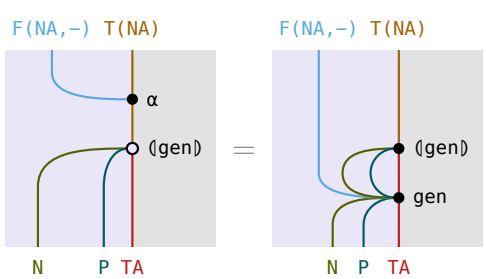
(A.3.3b)

```
gen ≜ F(𝟙,moves trans N(union)) zip N(cp(F(A,−)) E(α))
```

```
u = ((1,2,3,4),({[5]},{[6]},{[7]},{[8]}))      : F(N A,N(E(L A)))
F(𝟙,moves trans N(union))                        𝟙 keeps the column, the path SETS move
  moves({[5]},{[6]},{[7]},{[8]})
   = {({[6]},{[7]},{[8]},{[5]}),
      ({[5]},{[6]},{[7]},{[8]}),
      ({[8]},{[5]},{[6]},{[7]})}                 : E(N(E(L A)))  down, unmoved, up
  trans(that)
   = ({{[6]},{[5]},{[8]}},{{[7]},{[6]},{[5]}},
      {{[8]},{[7]},{[6]}},{{[5]},{[8]},{[7]}}) : N(E(E(L A)))  row k gets rows k-1, k, k+1
  N(union)(that)
   = ({[5],[6],[8]},{[5],[6],[7]},
      {[6],[7],[8]},{[5],[7],[8]})               : N(E(L A))     every path into row k, none dropped
zip(that)
   = ((1,{[5],[6],[8]}),(2,{[5],[6],[7]}),
      (3,{[6],[7],[8]}),(4,{[5],[7],[8]}))       : N(F(A,E(L A)))  each row: its square, and the
                                                                    paths it may be put in front of
N(cp P(α))(that)
   = ({[1,5],[1,6],[1,8]},{[2,5],[2,6],[2,7]},
      {[3,6],[3,7],[3,8]},{[4,5],[4,7],[4,8]}) : N(E(L A))     cp pairs the square with each path,
                                                               α prefixes it: α(1,[5])=[1,5]
```

§A.3.4. gen is an F-algebra; ⦇gen⦈: α⦇gen⦈=F(𝟙,⦇gen⦈)gen

(A.3.4a)



α puts the column back on the list, then the fold reads all of it — the fold reads the rest under F, then one gen puts the column in front

(A.3.4b)

```
xs = [(1,2,3,4),(5,6,7,8)] = α((1,2,3,4),[(5,6,7,8)])   : L(N A)

⦇gen⦈(xs) = gen((1,2,3,4), ⦇gen⦈[(5,6,7,8)])
```

both halves are in `gen ≜ F(𝟙,moves trans N(union)) zip N(cp(F(A,−)) E(α))` : the tail folds to `({[5]},{[6]},{[7]},{[8]})`, one path per row and each of them one square long, and `gen` on it is `gen ≜ F(𝟙,moves trans N(union)) zip N(cp(F(A,−)) E(α))` 's walk.

§A.3.5. Q : F(N A,N(L A))→N(L A), an F(N A,−)-algebra: Q=F(𝟙,moves trans N(est(R))) zip N(α)

(A.3.5a)

```
Q : F(N A,N(L A)) → N(L A)                    the fold's algebra: a new column, one path per row

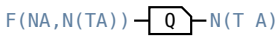
u = ((1,2,3,4),([5],[6],[7],[8]))             : F(N A,N(L A))
F(𝟙,moves trans N(est(R)))                      𝟙 keeps the column, the paths move
  moves([5],[6],[7],[8])
   = {([6],[7],[8],[5]),
      ([5],[6],[7],[8]),
      ([8],[5],[6],[7])}                      : E(N(L A))     down, unmoved, up
  trans(that)
   = ({[6],[5],[8]},{[7],[6],[5]},
      {[8],[7],[6]},{[5],[8],[7]})            : N(E(L A))     row k gets rows k-1, k, k+1
  N(est(R))(that)
   = ([5],[5],[6],[5])                        : N(L A)        the cheapest into each row —
                                                              chosen BEFORE the new square
zip(that)
   = ((1,[5]),(2,[5]),(3,[6]),(4,[5]))        : N(F(A,L A))   each row: its square, and the one
                                                              predecessor that survived
N(α)(that)
   = ([1,5],[2,5],[3,6],[4,5])                : N(L A)        α(1,[5])=[1,5]
```

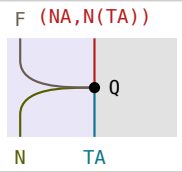
lax(moves); lax(trans); lax(zip); F(𝟙,moves trans N(est(R))) = [l,pm r]; zip = [N(inl),zip' N(inr)]; inl α = wrap; inr α = cons ⇒ Q = [N(wrap),pm zip' N(cons)] (A.3.5b)

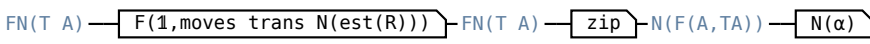
at the last column **Q** starts one path per row, and at each earlier one it puts each square in front of the cheapest of the three kept paths it can step to — the algebra **lax(moves); lax(trans); lax(zip); lax(setify); R² ⊑ R; F(𝟙,N(est(R))) Q ⊑ gen N(est(R)) ⇒ ⦇Q⦈ setify est(R) ⊑ paths est(R)** 's last step folds

circuit — the fork is F(NA,N(LA))=NA+NA×N(LA)

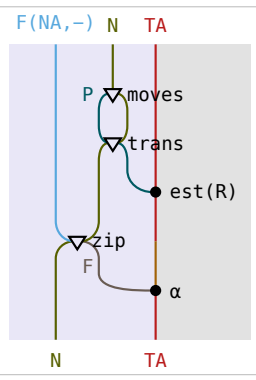
Hinze–Marsden

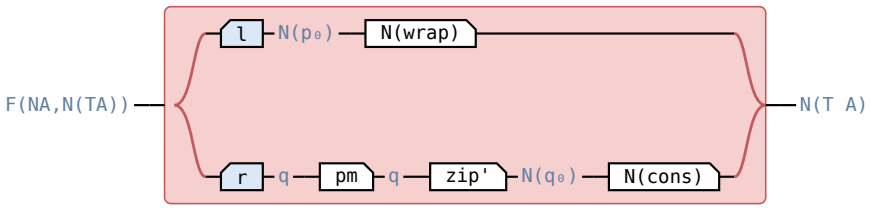






= the fusion condition lax(moves); lax(trans); lax(zip); R² ⊑ R; F(𝟙,est(R)) α ⊑ cp(F(A,−)) E(α) est(R) ⇒ F(𝟙,N(est(R))) Q ⊑ gen N(est(R)) read as a definition,

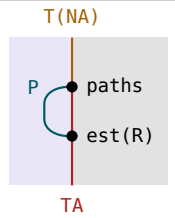
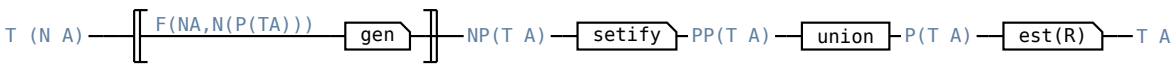
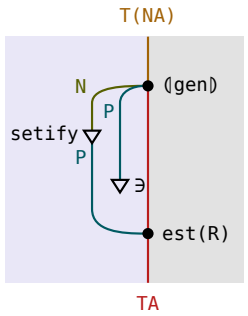
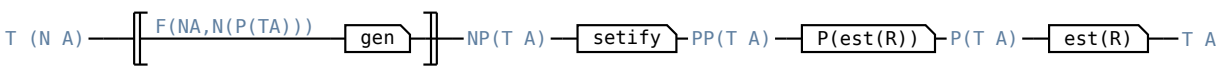
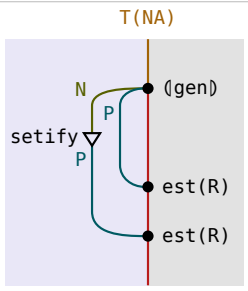
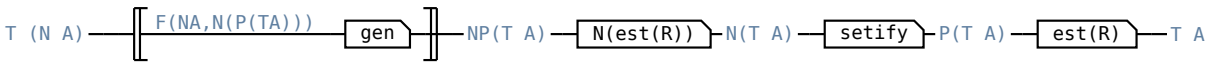
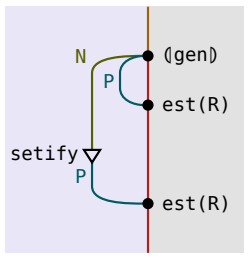
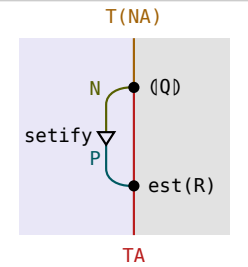


= 

zip=𝟙+zip', α=[τ,cons]

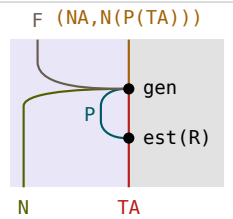
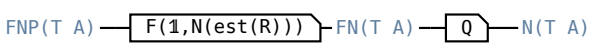
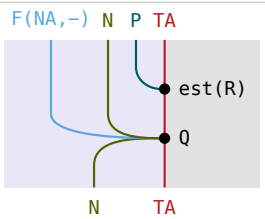
lax(moves); lax(trans); lax(zip); lax(setify); R² ⊑ R; F(𝟙,N(est(R))) Q ⊑ gen N(est(R)) (A.3.5c)
⇒ (Q) setify est(R) ⊑ paths est(R)

shortest paths on a cylinder: the dynamic programming approach to exhaustive search allows a path of least cost to be found in `O(n×m)` time; `Q` is `lax(moves); lax(trans); lax(zip); F(𝟙,moves trans N(est(R))) = [l,pm r]; zip = [N(inl),zip' N(inr)]; inl α = wrap; inr α = cons`
⇒ `Q = [N(wrap),pm zip' N(cons)]` 's algebra.

circuit	Hinze–Marsden
	
=  `paths ≜ (gen) setify union` at `paths`	
⊒  `P(est(R)) est(R)⊑union est(R)` — a minimum in each set, then a minimum of those; `R` transitive	
⊒  `setify` lax natural	
⊒  fusion at `lax(moves); lax(trans); lax(zip); R² ⊑ R; F(𝟙,est(R)) α ⊑ cp(F(A,–)) E(α) est(R)` `⇒ F(𝟙,N(est(R))) Q ⊑ gen N(est(R))`	

`lax(moves); lax(trans); lax(zip); R² ⊑ R; F(𝟙,est(R)) α ⊑ cp(F(A,−)) E(α) est(R)` (A.3.5d)
`⇒ F(𝟙,N(est(R))) Q ⊑ gen N(est(R))`

fusion: the condition for fusion in **`lax(moves); lax(trans); lax(zip); lax(setify); R² ⊑ R; F(𝟙,N(est(R))) Q ⊑ gen N(est(R))`** `⇒ (|Q|) setify est(R) ⊑ paths est(R)` 's last step, used to derive a definition of `Q`.

circuit	Hinze–Marsden
	
⊑  (7.13), then `zip`, `trans`, `moves` lax natural	

(7.13) is `F(𝟙,est(R))α⊑cp P(α) est(R)`, Theorem 7.1 `Map(f); 𝟙 ⊑ R` `⇒ f° F(R) f ⊑ R ⇔ F(est(R)) f ⊑` $\frac{\mathtt{F(∋)\ f}}{\mathtt{∋}}$ `est(R)` at the map `α` with $\frac{\mathtt{F(𝟙,∋)α}}{\mathtt{∋}}$ `=cp P(α)`: extending every path in a set and then taking a minimum is beaten by extending one minimum. It is the crux here, not the greedy theorem.